PRÁCTICA GUIADA · PRIMERA PRUEBA UNITARIA · PREPARACIÓN PARA JEST

# Primeras Pruebas

Documentación de la práctica: de ejecutar una función a comprobarla automáticamente

Un proyecto mínimo de Node.js con una sola función, `calcularDescuento`, que pasó de ejecutarse y revisarse a ojo a **comprobarse automáticamente contra resultados esperados**. En el camino se provocó un error lógico a propósito para demostrar que un programa puede ejecutarse sin fallar y aun así calcular mal, y que una comprobación automática lo detecta sin depender de quien mira la pantalla.

| | |
|---|---|
| **Integrante** | Víctor Benjamín Bautista Rojas (@Br-Benja) |
| **Materia** | Ingeniería de Software · Parcial 3 |
| **Práctica** | LAB 03-02 · Ejemplo guiado: cálculo de descuentos (previa al ToDo Refactor Lab) |
| **Tecnologías** | JavaScript (CommonJS) · Node.js v24.14.0 · npm 11.9.0 · Visual Studio Code · PowerShell |
| **Resultado** | 4 de 4 casos superados con `comprobar()` · el error lógico `/ 10` se detectó automáticamente en 3 de 4 casos |
| **Fecha** | 19 de octubre de 2026 |

# Contenido

1. Introducción y objetivo
2. Metodología de trabajo
3. Herramientas y tecnologías
4. Preparación del entorno (paso 0)
5. Estructura del proyecto
6. Desarrollo paso a paso
7. Casos de prueba
8. Anatomía de una prueba
9. Hacia Jest
10. Problemas encontrados y cómo se resolvieron
11. Checkpoint antes de Jest y del ToDo
12. Reflexión final
13. Conclusiones

Anexo A. Código final de porcentaje.js

Anexo B. Código final de probarPorcentaje.js

Anexo C. package.json

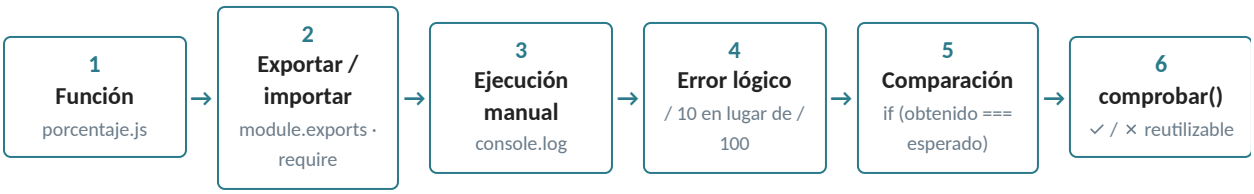
# 1. Introducción y objetivo

## 1.1 El reto

La guía parte de una pregunta sencilla: si una función “funciona”, ¿cómo lo sabemos? Ejecutarla y ver un número en la terminal no es lo mismo que **comprobar que ese número es el correcto**. Esta práctica construye, sin ninguna biblioteca de pruebas, el mecanismo mínimo que después formaliza Jest: ejecutar una unidad de código, conocer el resultado esperado y dejar que el programa decida si coinciden.

## 1.2 Objetivo

Crear desde cero un proyecto con dos archivos (`porcentaje.js`, que contiene la función, y `probarPorcentaje.js`, que la ejecuta), definir resultados esperados, provocar un error lógico y construir una comprobación automática reutilizable antes de conocer Jest.




> **Pregunta guía:** ¿quién tuvo que detectar que 0, -300 y -900 eran incorrectos? **Respuesta:** en la ejecución manual, yo, comparando cada número con la tabla. Con 50 o 500 casos esa revisión sería lenta y fácil de equivocar; por eso la práctica termina con una función que compara por mí.

## 1.3 Resultado final

- `calcularDescuento(precio, porcentaje)` vive en `porcentaje.js` y se exporta con `module.exports`.
- `probarPorcentaje.js` la importa con `require("./porcentaje")` y la comprueba con cuatro casos: dos normales, uno frontera y uno extremo.
- Con la fórmula correcta los cuatro casos muestran ✓; con el error `/ 10`, `comprobar()` marca 3 × e indica el valor esperado y el obtenido.

- Se identificaron las seis piezas de una prueba (entrada, unidad, obtenido, esperado, comparación y resultado) y su equivalente en Jest.

# 2. Metodología de trabajo

La práctica se realizó de forma **individual** en una computadora con Windows, siguiendo la guía en orden y sin adelantar pasos: cada archivo se escribió solo cuando la guía lo indicaba y cada ejecución se comparó con el resultado esperado antes de continuar.

## 2.1 Rutina de cada paso

1. **Leer:** qué cambia en el paso y qué resultado se espera, antes de escribir código.
2. **Editar:** escribir el código en VS Code y guardar (Ctrl + S).
3. **Ejecutar:** `node .\probarPorcentaje.js` en la terminal integrada (PowerShell), dentro de `primeras-pruebas`.
4. **Comparar:** contrastar la salida con la tabla de resultados esperados del paso 5.
5. **Evidencia:** captura con el editor y la terminal en la misma imagen.

## 2.2 Reglas que apliqué

- Primero se define el resultado esperado y después se ejecuta.
- Una sola modificación a la vez: en el paso 7 solo cambió `porcentaje.js`; `probarPorcentaje.js` quedó igual, así que la diferencia en la salida solo podía venir del error.
- Vista dividida en VS Code: `porcentaje.js` a la izquierda y `probarPorcentaje.js` a la derecha, para que cada captura muestre la función, la prueba y el resultado juntos.
- Trabajar siempre dentro de `primeras-pruebas`; el prompt de PowerShell lo confirma en cada captura.

# 3. Herramientas y tecnologías

| Herramienta | Para qué la usé |
|---|---|
| **Node.js v24.14.0** | Ejecutar JavaScript fuera del navegador: `node probarPorcentaje.js`. |
| **npm 11.9.0** | Crear `package.json` con `npm init -y`. Todavía no se instala ninguna dependencia. |
| **JavaScript (CommonJS)** | `module.exports` para exportar y `require()` para importar entre archivos; `===` y plantillas de texto para comparar y reportar. |
| **Visual Studio Code** | Editor con vista dividida y terminal integrada. |
| **PowerShell (Windows)** | Terminal donde se ejecutaron todos los comandos. |

# 4. Preparación del entorno (paso 0)

La guía no requiere Vite, Vitest ni un servidor web: el JavaScript se ejecuta directamente con Node.js.

## 4.1 Verificar Node.js y npm

```
node --version
npm --version
```

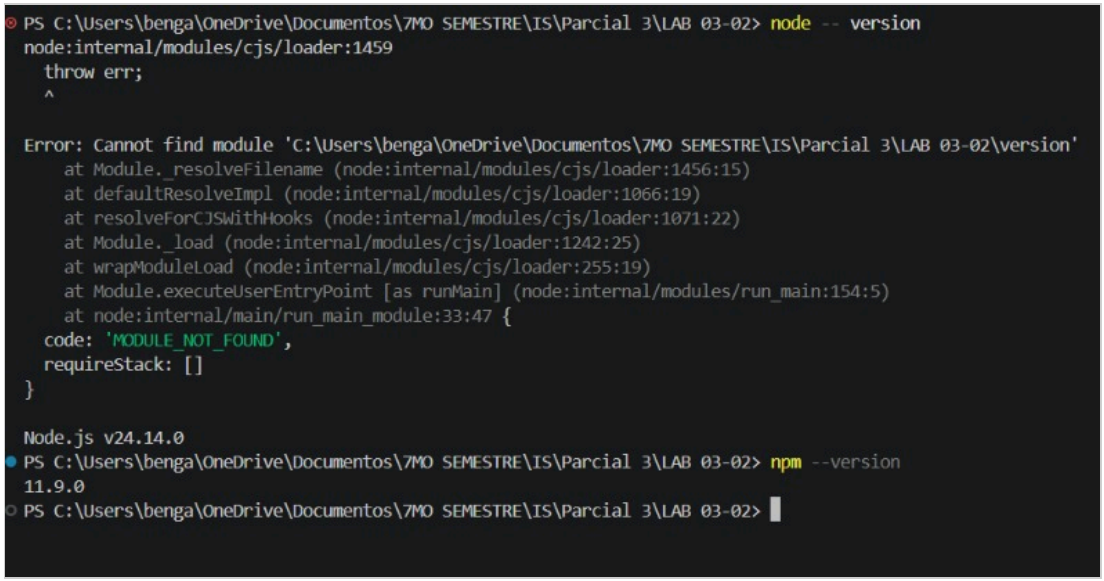



***Figura 1.*** *Paso 0.1 · primer intento con* `node -- version` *(error* `MODULE_NOT_FOUND`*) y* `npm --version` *respondiendo 11.9.0.*

> **Incidencia:** el primer intento se escribió `node -- version`, con un espacio. Node interpretó `--` como el fin de sus opciones y `version` como el nombre de un archivo a ejecutar, por eso respondió `Error: Cannot find module '...\LAB 03-02\version'`. La línea `Node.js v24.14.0` del final no es la respuesta al comando: Node imprime su versión al pie de cualquier error fatal. Escrito correctamente, `node --version` respondió `v24.14.0` (Figura 2).

> **Lección:** en la terminal un espacio cambia el significado: `--version` es una opción; `-- version` significa «se acabaron las opciones, ejecuta el archivo *version*».

## 4.2 Crear e inicializar la carpeta

```
mkdir primeras-pruebas
cd primeras-pruebas
npm init -y
```

```
>> mkdir primeras-pruebas
>> cd primeras-pruebas
>> npm init -y         # crea package.json
v24.14.0
11.9.0


    Directorio: C:\Users\benga\OneDrive\Documentos\7MO SEMESTRE\IS\Parcial 3\LAB 03-02


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----     19/10/2026  12:19 p. m.                primeras-pruebas
Wrote to C:\Users\benga\OneDrive\Documentos\7MO SEMESTRE\IS\Parcial 3\LAB 03-02\primeras-pruebas\package.json:

{
  "name": "primeras-pruebas",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}
```

***Figura 2.*** *Paso 0.2 · los comandos se pegaron en bloque (PowerShell muestra* `>>` *en cada línea):* `v24.14.0`, `11.9.0`, *se crea* `primeras-pruebas` *y* `npm init -y` *escribe* `package.json`.

> **Observación:** npm 11 agrega `"type": "commonjs"` al `package.json`; la guía no lo muestra porque versiones anteriores de npm no lo incluían. Ese campo le indica a Node que trate los `.js` como módulos CommonJS, justo el sistema de `module.exports` y `require()`. En el ToDo Refactor Lab se usa `"type": "module"`, por eso allá se escribe `import` / `export`.

## 4.3 Crear los archivos

En VS Code se abrió la carpeta y se crearon `porcentaje.js` y `probarPorcentaje.js`.

```
PS C:\Users\benga\OneDrive\Documentos\7MO SEMESTRE\IS\Parcial 3\LAB 03-02> cd .\primeras-pruebas\
PS C:\Users\benga\OneDrive\Documentos\7MO SEMESTRE\IS\Parcial 3\LAB 03-02\primeras-pruebas> ls


    Directorio: C:\Users\benga\OneDrive\Documentos\7MO SEMESTRE\IS\Parcial 3\LAB 03-02\primeras-pruebas


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----     19/10/2026  12:19 p. m.            252 package.json
-a----     19/10/2026  12:22 p. m.            146 porcentaje.js
-a----     19/10/2026  12:22 p. m.            224 probarPorcentaje.js


PS C:\Users\benga\OneDrive\Documentos\7MO SEMESTRE\IS\Parcial 3\LAB 03-02\primeras-pruebas>
```

***Figura 3.*** *Paso 0.3 ·* `ls` *dentro de* `primeras-pruebas`: `package.json`, `porcentaje.js` *y* `probarPorcentaje.js`.

# 5. Estructura del proyecto

```
primeras-pruebas/
├── package.json          Configuración básica (npm init -y)
├── porcentaje.js         La unidad a probar: calcularDescuento
├── probarPorcentaje.js   Ejecuta y comprueba la función
└── evidencias/           Capturas de cada paso
```

## 5.1 Responsabilidad de cada archivo

| Archivo | Responsabilidad | Cómo se usó en la práctica |
|---|---|---|
| `package.json` | Configuración básica del proyecto. | Creado con `npm init -y`; nombre `primeras-pruebas` y `"type": "commonjs"`. |
| `porcentaje.js` | La unidad a probar: `calcularDescuento`. | Solo calcula y exporta; no imprime nada ni conoce los casos de prueba. Es el único archivo que se modificó para provocar el error. |
| `probarPorcentaje.js` | Ejecutar y comprobar la función. | Evolucionó de un `console.log` a un `if` y después a la función `comprobar()` con cuatro casos. |

> **Decisión:** Necesitamos separar la función de su prueba **porque** así la función se puede usar en cualquier parte sin arrastrar código de prueba, y las pruebas pueden crecer o cambiar sin tocar la lógica que protegen.

# 6. Desarrollo paso a paso

En esta sección se documenta cada paso de la guía en el orden en que se hizo: qué archivo cambió, qué comando se ejecutó, el código, la evidencia y lo que demuestra.

## Paso 1 · Escribir la función en porcentaje.js

| | |
|---|---|
| **Archivo** | `porcentaje.js` |
| **Comando** | Ninguno: todavía no se ejecuta |
| **Evidencia** | Figura 4 |

```
function calcularDescuento(precio, porcentaje) {
  return precio - (precio * porcentaje / 100);
}
```

La función recibe el precio original y el porcentaje de descuento. Para 100 y 10 calcula 100 − (100 × 10 / 100) = **90**.

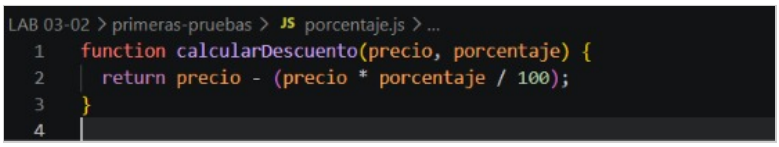



***Figura 4.*** *Paso 1 · `calcularDescuento` definida en `porcentaje.js`.*

## Paso 2 · Exportar la función

| | |
|---|---|
| **Archivo** | `porcentaje.js` |
| **Comando** | Ninguno |
| **Evidencia** | Figura 5 |

```
function calcularDescuento(precio, porcentaje) {
  return precio - (precio * porcentaje / 100);
}

module.exports = { calcularDescuento };
```

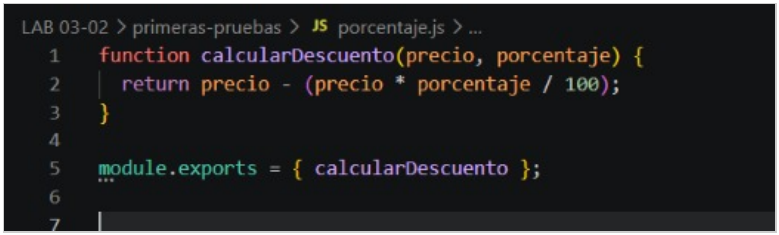



***Figura 5.*** *Paso 2 · `porcentaje.js` completo, con `module.exports`.*

> **Decisión:** Necesitamos `module.exports` **porque** sin él la función solo existe dentro de `porcentaje.js`. Exportarla la hace disponible para otros archivos; la función no se copia, sigue viviendo en su archivo.

## Paso 3 · Importar la función en probarPorcentaje.js

| | |
|---|---|
| **Archivo** | `probarPorcentaje.js` |
| **Comando** | Ninguno |
| **Evidencia** | Figura 6 |

```
const { calcularDescuento } = require("./porcentaje");
```

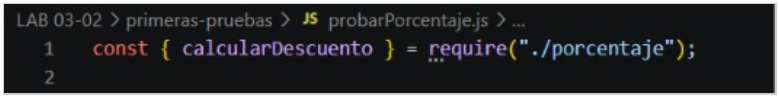



***Figura 6.*** *Paso 3 · `require("./porcentaje")` en `probarPorcentaje.js`.*

`require("./porcentaje")` carga lo que `porcentaje.js` exportó. El prefijo `./` significa “en esta misma carpeta” y Node completa la extensión `.js` por su cuenta.

## Paso 4 · Primera ejecución

| | |
|---|---|
| **Archivo** | `probarPorcentaje.js` |
| **Comando** | `node .\probarPorcentaje.js` |
| **Resultado** | **90** |
| **Evidencia** | Figura 7 |

```
const { calcularDescuento } = require("./porcentaje");

const resultado = calcularDescuento(100, 10);
console.log(resultado);
```

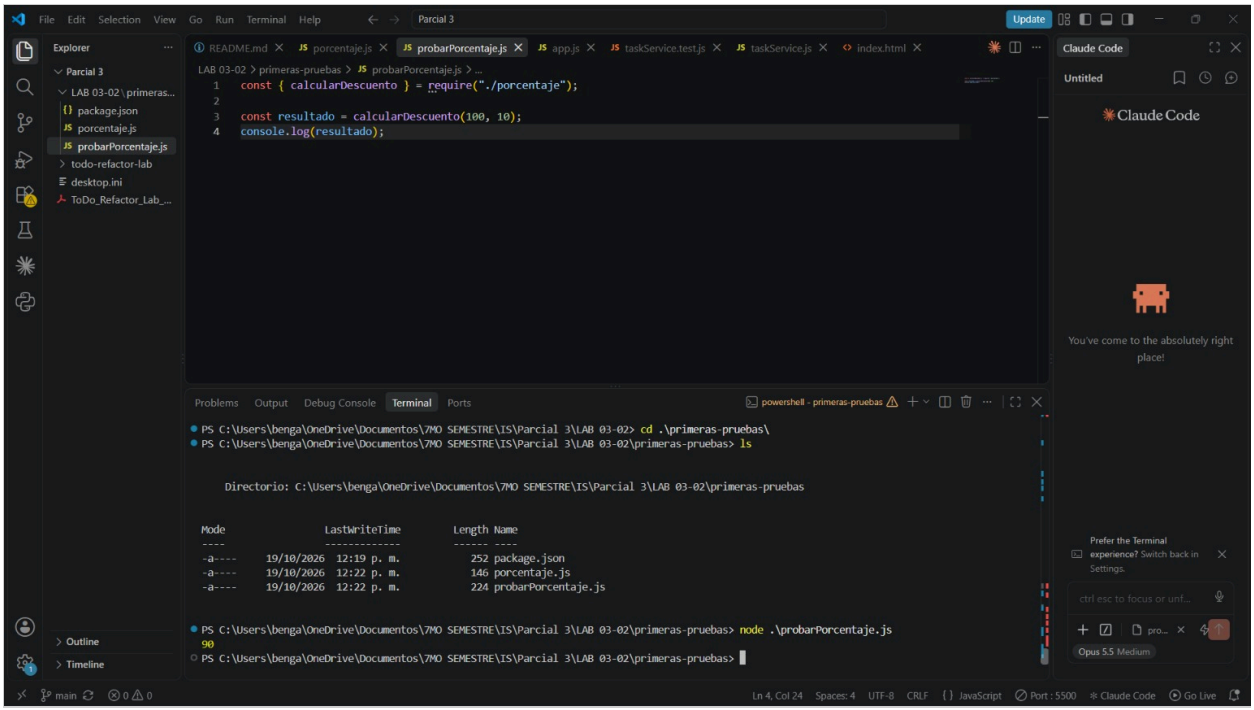



***Figura 7.*** *Paso 4 · Checkpoint 1: la terminal muestra* `90`.

> **Checkpoint 1:** la salida `90` confirma que la función se implementó en `porcentaje.js`, se exportó, se importó desde `probarPorcentaje.js` y se ejecutó correctamente. Todavía **no** es una prueba automatizada: soy yo quien sabe que 90 es lo correcto.

## Paso 5 · Definir casos y resultados esperados

Antes de ejecutar más valores se fijó qué debe devolver la función. Esta tabla es la referencia para detectar errores en los pasos siguientes.

| Caso | Entrada (precio, %) | Resultado esperado | Tipo |
|---|---|---|---|
| 1 | 100, 10 | 90 | Normal |
| 2 | 200, 25 | 150 | Normal |
| 3 | 100, 0 | 100 | Frontera: sin descuento |
| 4 | 100, 100 | 0 | Extremo: descuento total |

## Paso 6 · Ejecutar los cuatro casos manualmente

| | |
|---|---|
| **Archivo** | `probarPorcentaje.js` |
| **Comando** | `node .\probarPorcentaje.js` |
| **Resultado** | 90 · 150 · 100 · 0 |
| **Evidencia** | Figura 8 |

```
const { calcularDescuento } = require("./porcentaje");

console.log(calcularDescuento(100, 10));
console.log(calcularDescuento(200, 25));
console.log(calcularDescuento(100, 0));
console.log(calcularDescuento(100, 100));
```

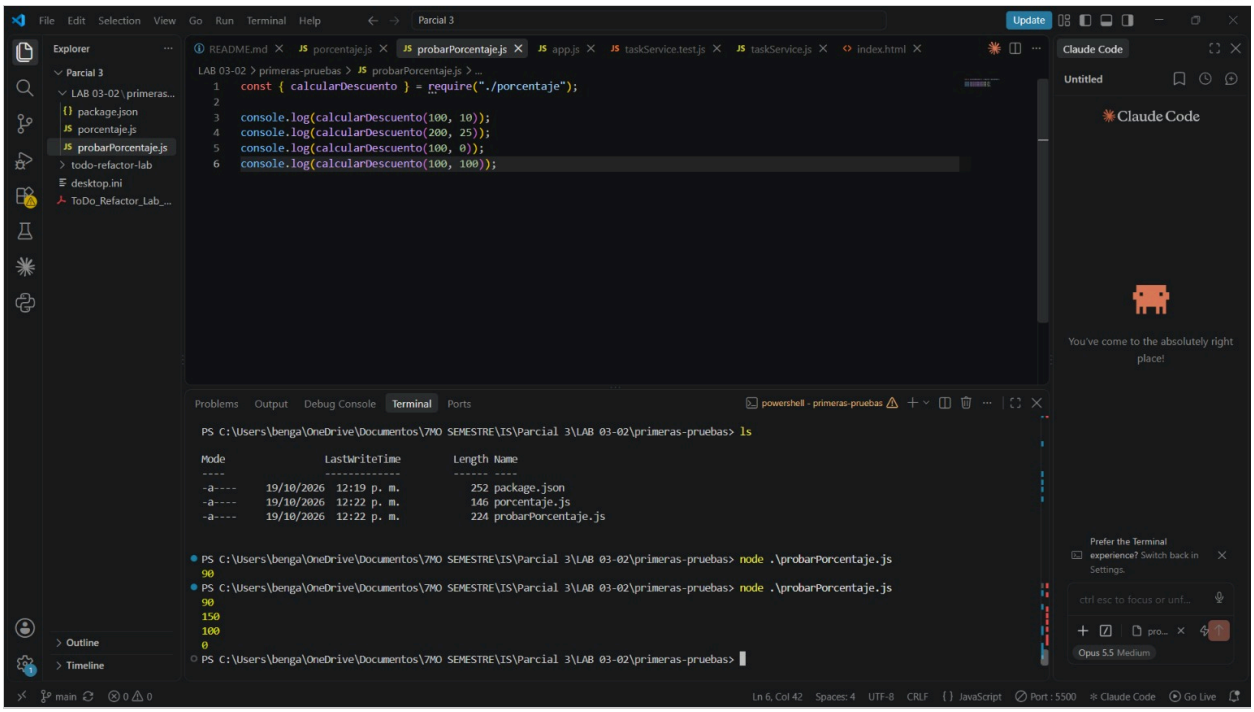



***Figura 8.*** *Paso 6 · salida `90`, `150`, `100` y `0`: coincide con la tabla del paso 5.*

> **Comprobación manual:** Node ejecuta los cuatro casos, pero la comparación con la tabla la hice yo, revisando cada número.

## Paso 7 · Provocar un error lógico

| | |
|---|---|
| **Archivo** | `porcentaje.js` (`probarPorcentaje.js` no se tocó) |
| **Cambio** | `/ 100` → `/ 10` |
| **Resultado** | 0 · -300 · 100 · -900 |
| **Evidencia** | Figura 9 |

```
function calcularDescuento(precio, porcentaje) {
  return precio - (precio * porcentaje / 10); // ERROR INTENCIONAL
}

module.exports = { calcularDescuento };
```

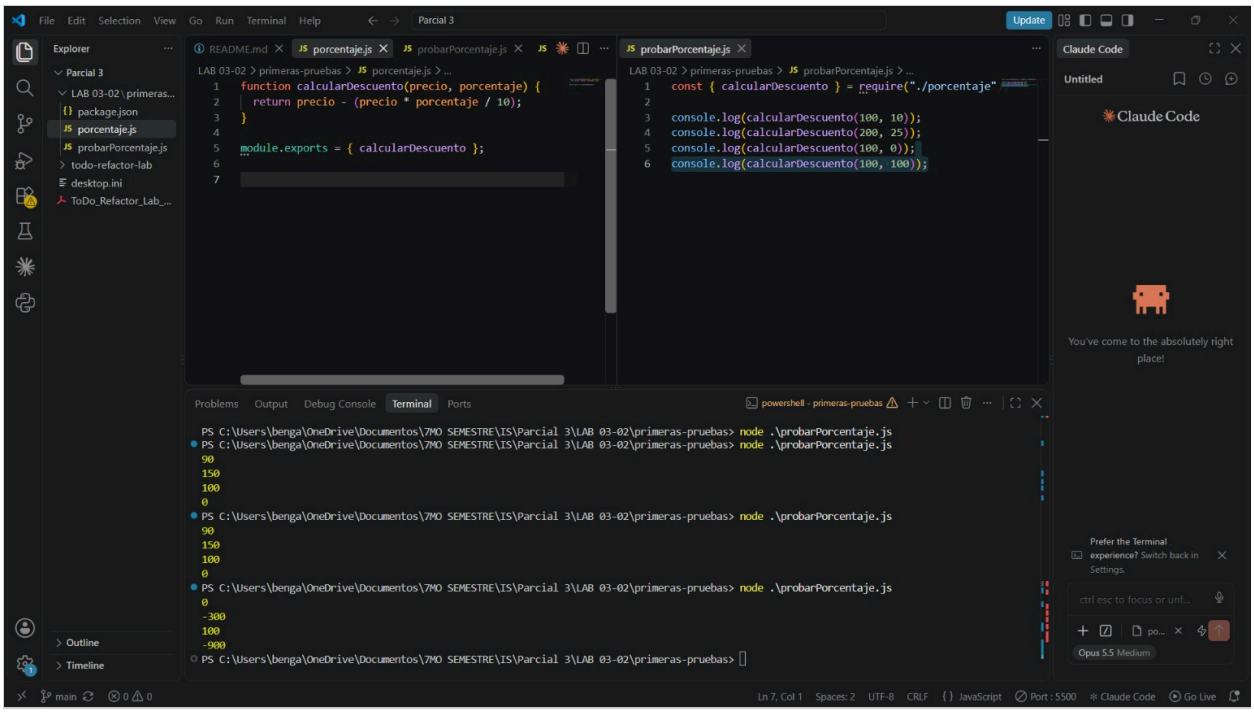



***Figura 9.*** *Paso 7 · con* `/ 10` *la misma prueba muestra* `0`, `-300`, `100` *y* `-900`.

| Entrada | Esperado | Obtenido | ¿Correcto? |
|---|---|---|---|
| 100, 10 | 90 | 0 | No |
| 200, 25 | 150 | -300 | No |
| 100, 0 | 100 | 100 | Sí |
| 100, 100 | 0 | -900 | No |

> **Error lógico:** Node no reportó ningún error porque `precio * porcentaje / 10` es una expresión válida. El programa se ejecuta y aun así produce resultados incorrectos. El caso `100, 0` incluso sale bien (0 entre cualquier número da 0), así que un solo caso no habría bastado para descubrir el problema.

## Paso 8 · Hacer que el programa compare por mí

| | |
|---|---|
| **Archivo** | `probarPorcentaje.js` |
| **Comando** | `node .\probarPorcentaje.js` |
| **Resultado** | `PRUEBA FALLIDA` con `/ 10` · `PRUEBA SUPERADA` con `/ 100` |
| **Evidencia** | Figuras 10 y 11 |

```
const { calcularDescuento } = require("./porcentaje");

const resultado = calcularDescuento(100, 10);

if (resultado === 90) {
  console.log("PRUEBA SUPERADA");
} else {
  console.log("PRUEBA FALLIDA");
}
```

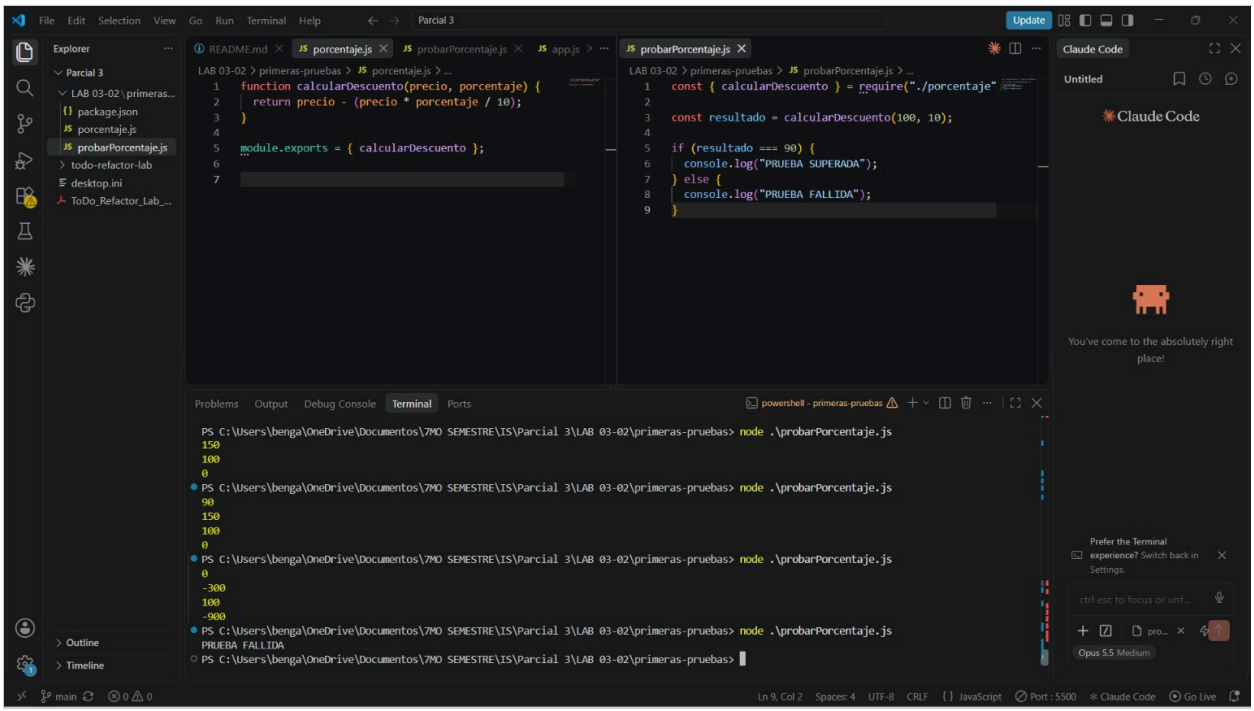



***Figura 10.*** *Paso 8 · con el error `/ 10` todavía presente aparece `PRUEBA FALLIDA`.*

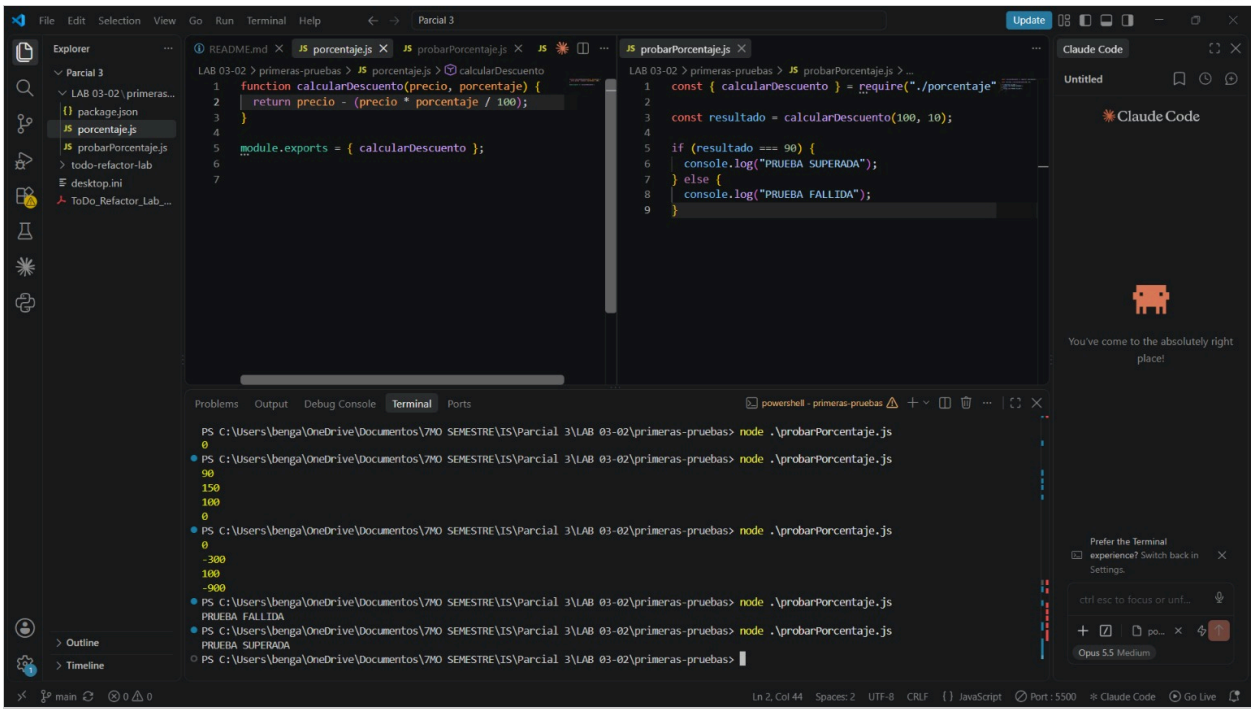



***Figura 11.*** *Paso 8 · al restaurar* `/ 100` *aparece* `PRUEBA SUPERADA`.

> **Cambio fundamental:** ya no soy yo quien compara mentalmente el resultado. El programa ejecuta la función, conoce el valor esperado (90) y decide automáticamente si coincide.

## Paso 9 · Convertir la comprobación en algo reutilizable

| | |
|---|---|
| **Archivo** | `probarPorcentaje.js` |
| **Comando** | `node .\probarPorcentaje.js` |
| **Resultado** | 4 ✓ con `/ 100` · 3 × y 1 ✓ con `/ 10` |
| **Evidencia** | Figuras 12 y 13 |

```
const { calcularDescuento } = require("./porcentaje");

function comprobar(descripcion, obtenido, esperado) {
  if (obtenido === esperado) {
    console.log(`✓ ${descripcion}`);
  } else {
    console.log(`✗ ${descripcion} | esperado: ${esperado}, obtenido: ${obtenido}`);
  }
}

comprobar("10% de 100", calcularDescuento(100, 10), 90);
comprobar("25% de 200", calcularDescuento(200, 25), 150);
comprobar("0% de 100", calcularDescuento(100, 0), 100);
comprobar("100% de 100", calcularDescuento(100, 100), 0);
```

La guía pide ejecutar esta versión con la fórmula correcta. Además, como experimento, volví a poner `/ 10` para ver cómo reporta `comprobar()` los fallos: cada ✗ indica qué esperaba y qué obtuvo.

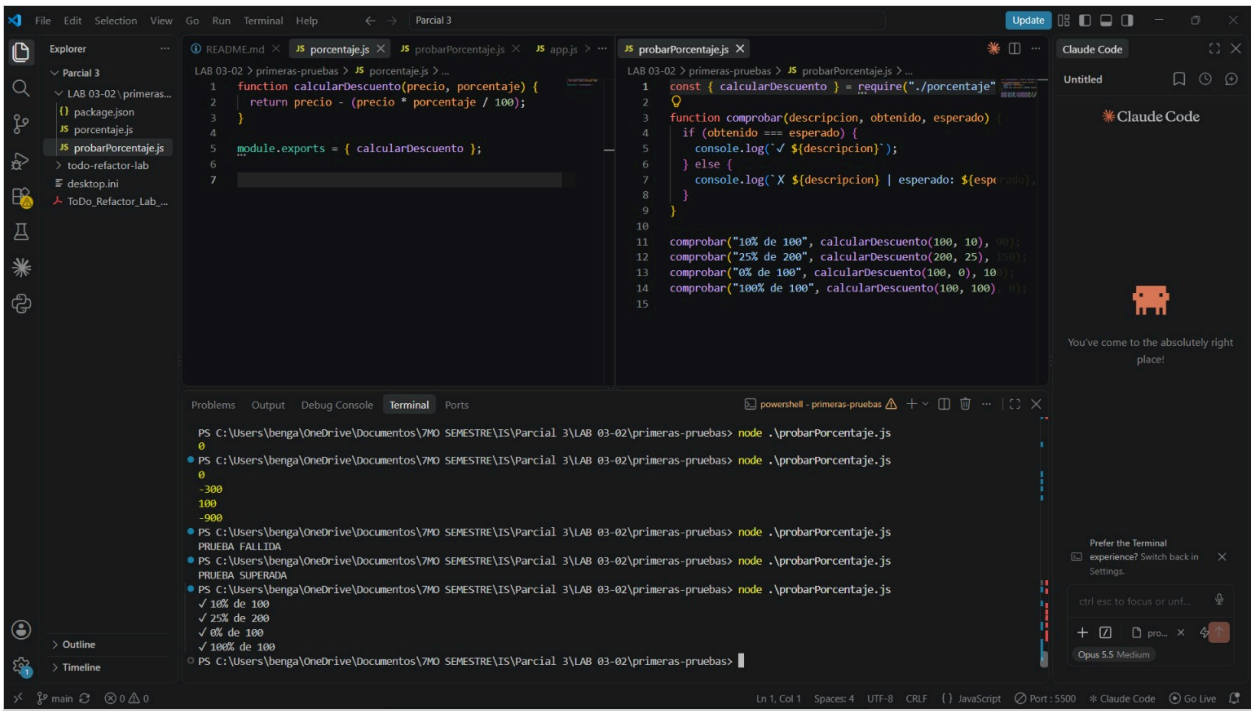



***Figura 12.*** *Paso 9 · con `/ 100`: los cuatro casos muestran ✓.*

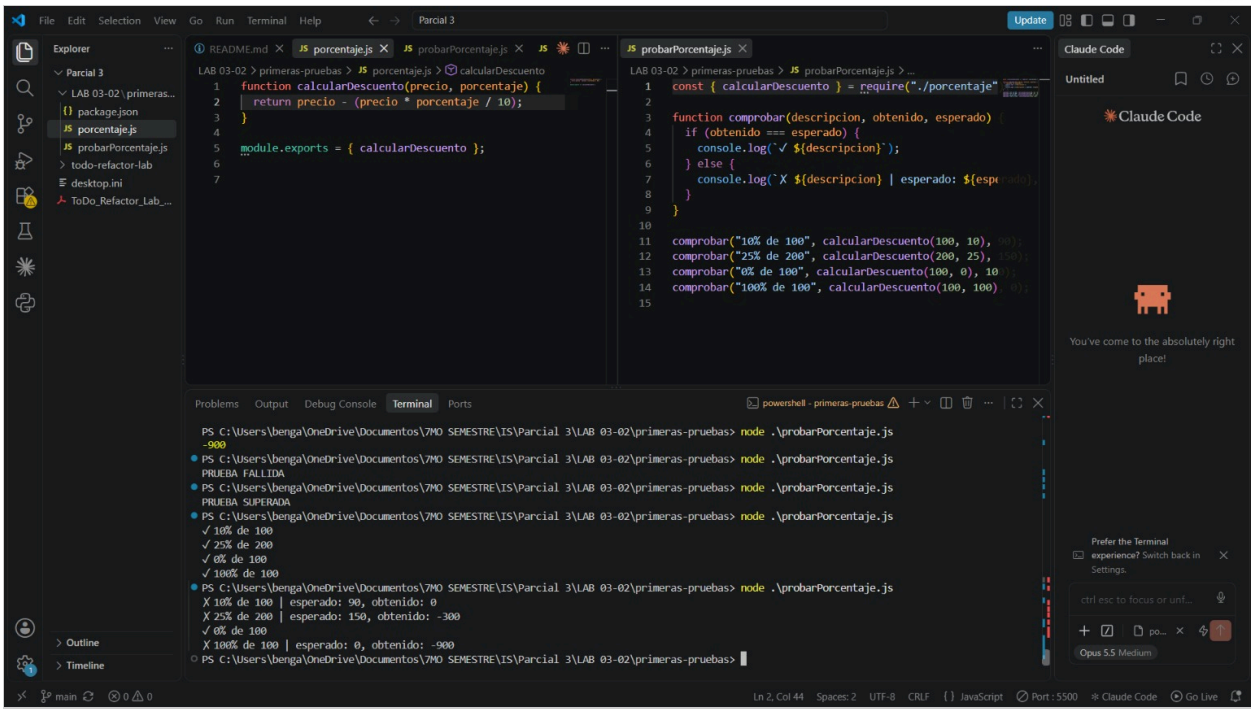



***Figura 13.*** *Paso 9 · experimento con* `/ 10` *:* `comprobar()` *marca 3 ✕ con esperado y obtenido; solo el caso frontera* `0% de 100` *pasa.*

> **Ya construí un pequeño mecanismo de testing:** `comprobar()` hace comparaciones repetibles y, cuando falla, explica la diferencia. Lo que le falta es lo que da un framework: agrupar casos, contar cuántos pasan y fallan, y un reporte final.

# 7. Casos de prueba

Resumen de los cuatro casos de la guía con la fórmula correcta y con el error intencional.

| # | Entrada | Tipo | Esperado | Obtenido (/ 100) | Obtenido (/ 10) | comprobar() con / 10 |
|---|---|---|---|---|---|---|
| 1 | 100, 10 | Normal | 90 | 90 | 0 | ✕ |
| 2 | 200, 25 | Normal | 150 | 150 | -300 | ✕ |
| 3 | 100, 0 | Frontera | 100 | 100 | 100 | ✓ |
| 4 | 100, 100 | Extremo | 0 | 0 | -900 | ✕ |

> **Por qué varios casos:** el caso 3 pasa incluso con la fórmula equivocada, porque con 0 % de descuento el divisor no importa. Si solo hubiera probado ese caso, el error habría pasado desapercibido. Los casos normales y el extremo fueron los que lo revelaron.

# 8. Anatomía de una prueba

| Elemento | Ejemplo | Pregunta |
|---|---|---|
| Entrada | `100, 10` | ¿Con qué datos pruebo? |
| Unidad | `calcularDescuento(...)` | ¿Qué comportamiento ejecuto? |
| Obtenido | `90` | ¿Qué produjo el código? |
| Esperado | `90` | ¿Qué debería producir? |
| Comparación | `obtenido === esperado` | ¿Coinciden? |
| Resultado | PASS / FAIL (✓ / ×) | ¿La comprobación pasó? |

**ENTRADA → EJECUCIÓN → OBTENIDO ↔ ESPERADO → PASS / FAIL**

En `comprobar("10% de 100", calcularDescuento(100, 10), 90)` están todas las piezas: la descripción nombra el caso, `calcularDescuento(100, 10)` es la ejecución con su entrada, `90` es el esperado y la función compara y reporta.

# 9. Hacia Jest

Jest formaliza la misma idea. Más adelante `probarPorcentaje.js` será sustituido por `porcentaje.test.js`, y una prueba equivalente se verá así:

```
test("aplica correctamente un descuento del 10%", () => {
  expect(calcularDescuento(100, 10)).toBe(90);
});
```

| Ahora | Con Jest |
|---|---|
| `probarPorcentaje.js` | `porcentaje.test.js` |
| `comprobar(...)` | `test(...)` |
| `obtenido === esperado` | `expect(obtenido).toBe(esperado)` |
| `node probarPorcentaje.js` | `npm test`, cuando Jest esté configurado |

> **Idea clave:** Jest no inventa una idea diferente: automatiza y organiza la misma relación entre ejecución, resultado obtenido y resultado esperado que construí a mano con `comprobar()`.

# 10. Problemas encontrados y cómo se resolvieron

| Problema | Causa | Solución |
|---|---|---|
| `node -- version` respondió `Cannot find module ...\version` (`MODULE_NOT_FOUND`). | El espacio después de `--`: Node lo tomó como fin de opciones y buscó un archivo llamado `version`. | Escribir la opción junta: `node --version` → `v24.14.0`. |
| Al pegar varios comandos, PowerShell mostró `>>` en cada línea y volvió a imprimir las versiones. | PowerShell recibió el bloque pegado como una entrada de varias líneas y ejecutó todo en orden, incluidos los comandos de verificación. | No era un error: la carpeta y el package.json se crearon bien. Para evidencias más limpias conviene ejecutar un comando a la vez. |
| `mkdir` imprimió una tabla con `Directorio`, `Mode` y `LastWriteTime`. | En PowerShell `mkdir` usa `New-Item`, que muestra el elemento creado. | Es solo información; no requiere acción. |
| `package.json` distinto al de la guía: incluye `"type": "commonjs"`. | npm 11 agrega ese campo por defecto. | No requiere cambios; confirma que `require` / `module.exports` son el sistema correcto para este proyecto. |

> **Nota:** el resultado incorrecto del paso 7 no aparece aquí porque fue un error provocado a propósito; está documentado en la sección 6.

# 11. Checkpoint antes de Jest y del ToDo

**1. ¿Por qué porcentaje.js y probarPorcentaje.js están separados?**

**Respuesta:** Para separar el código que hace el trabajo (la función) del código que lo comprueba. Así la función se puede reutilizar en cualquier parte y las pruebas se pueden cambiar o ampliar sin tocar la lógica.

**2. ¿Qué hace module.exports?**

**Respuesta:** Hace pública la función calcularDescuento para que otros archivos del proyecto puedan importarla. La función sigue viviendo en porcentaje.js; no se copia.

**3. ¿Qué hace require("./porcentaje")?**

**Respuesta:** Carga lo que porcentaje.js exportó. El ./ indica “en esta misma carpeta” y Node agrega la extensión .js automáticamente.

**4. ¿Cuál es la diferencia entre ejecutar la función y comprobarla?**

**Respuesta:** Ejecutarla solo produce un resultado (por ejemplo, imprime 90) y una persona tiene que revisarlo. Comprobarla significa comparar ese resultado con un valor esperado y que el programa decida por sí mismo si pasó o falló.

**5. ¿Por qué /10 fue un error lógico que Node no reportó?**

**Respuesta:** Porque precio * porcentaje / 10 es una expresión válida en JavaScript: no hay error de sintaxis ni de ejecución. El programa corre, pero la fórmula ya no calcula un porcentaje, así que los resultados (0, -300, -900) son incorrectos. Solo una comprobación contra valores esperados lo detecta.

**6. ¿Qué trabajo realizará Jest que hoy realiza comprobar()?**

**Respuesta:** Comparar lo obtenido con lo esperado (expect(obtenido).toBe(esperado)) y reportar PASS/FAIL, además de encontrar los archivos de prueba, agruparlos, contar cuántos pasan o fallan y mostrar un reporte detallado con npm test.

# 12. Reflexión final

**¿Quién tuvo que detectar los errores?** En el paso 7 fui yo: Node imprimió 0, -300, 100 y -900 sin ninguna advertencia, y solo al compararlos con la tabla del paso 5 supe que estaban mal. Con cuatro casos es manejable; con 50 o 500 sería lento y muy fácil pasar uno por alto.

**Lo que cambió con la comprobación automática.** Desde el paso 8 el resultado esperado vive en el código. Con `comprobar()` basta ejecutar un comando para saber qué casos pasan y, cuando uno falla, qué esperaba y qué obtuvo. Romper la fórmula otra vez en el paso 9 lo dejó claro: los tres fallos aparecieron solos, con su explicación.

**Lo que me llevo para el ToDo Refactor Lab.** Una prueba solo da confianza si sé qué comportamiento protege. Por eso conviene escribir primero el resultado esperado, probar casos de distinto tipo (normales, frontera y extremos) y ver fallar la prueba al menos una vez para confirmar que realmente detecta el error.

# 13. Conclusiones

- Se completó el recorrido de la guía: `porcentaje.js` → exportar → importar → ejecutar → definir esperado → comparar → PASS/FAIL.
- Un programa puede ejecutarse sin errores y aun así ser incorrecto; los errores lógicos solo se detectan comparando contra un resultado esperado.
- Elegir bien los casos importa: el caso frontera `100, 0` pasó incluso con la fórmula equivocada.
- `comprobar()` es un mini framework de pruebas hecho a mano; Jest ofrece la misma idea, más organizada y con reportes.
- Los problemas del entorno (`node -- version`, el pegado en bloque en PowerShell y el `"type": "commonjs"` de npm 11) se entendieron y resolvieron sin afectar el resultado.

# Anexo A. Código final de porcentaje.js

```
function calcularDescuento(precio, porcentaje) {
  return precio - (precio * porcentaje / 100);
}

module.exports = { calcularDescuento };
```

# Anexo B. Código final de probarPorcentaje.js

```
const { calcularDescuento } = require("./porcentaje");

function comprobar(descripcion, obtenido, esperado) {
  if (obtenido === esperado) {
    console.log(`✓ ${descripcion}`);
  } else {
    console.log(`✗ ${descripcion} | esperado: ${esperado}, obtenido: ${obtenido}`);
  }
}

comprobar("10% de 100", calcularDescuento(100, 10), 90);
comprobar("25% de 200", calcularDescuento(200, 25), 150);
comprobar("0% de 100", calcularDescuento(100, 0), 100);
comprobar("100% de 100", calcularDescuento(100, 100), 0);
```

# Anexo C. package.json

```
{
  "name": "primeras-pruebas",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}
```